

オープンソースLLMの現在地 2026

“性能の追いつきと損益分岐——*self-host* か フロンティアか の意思決定 ”

- テックリーダー・EM 向け
- OSSモデルはどこまでフロンティアに迫ったのか
- 採用判断の軸を「性能」から「経済性」へ更新する

コーディングではOSSがフロンティアに追いついた

“争点は”性能差”から”損益分岐”へ移った — 「どちらが賢いか」ではなく「どこで自前化が得か」

39

2024～2025

論点：性能差

「OSSはどこまで賢いか」
ベンチのスコアを比較

2026

論点：損益分岐

「self-host が得になる境界は？」
コスト・運用・ライセンスで判断
トークン単価とトラフィック量が鍵

- SWE-bench では OSS 最上位が Claude Opus 4.6 の 0.6pt 差まで接近
- 実務の判断材料はスコアではなく「トラフィック量 × 単価 × 運用コスト」

本日の道筋：状況 → 複雑化 → 問い → 答え

“性能が並んだ結果、意思決定の軸そのものが変わったことを4ステップで辿る”



- 前半：状況と複雑化（性能・コスト・ライセンスの俯瞰）
- 後半：主要モデルの詳解と MoE が変える運用の現実

状況：ギャップはどこまで縮んだか

- 性能・コスト・ライセンスの3面から、2026年のOSSの実力を俯瞰する

性能ギャップは実質消滅：OSS最上位がOpus 4.6に肉薄

“ SWE-bench で MiniMax M2.5 = 80.2% — フロンティア Opus 4.6 = 80.8% との差はわずか 0.6pt ”

SWE-bench Verified (コード修正タスク成功率 / %)



紫 = フロンティア (クローズド) / それ以外 = オープンソース

- 上位モデル間の差はもはや誤差レベル — コード領域では実用差が消えた
- 「賢さ」で選別できないなら、次に効くのは経済性とライセンス

GLM-5がArena Elo 1451でオープンモデル首位に立った

“人間の対戦評価でも上位クローズドと同一レンジー 開放モデルが「体感品質」でも並んだ”

LMarena Elo (人間による総合対戦評価)



緑 = 開放モデル首位。上位はクローズドと同一レンジに収束

- ベンチだけでなくブラインド対戦でもトップ集団に開放モデルが入った
- 「クローズドでなければ品質が出ない」という前提は崩れた

2026年のOSSモデル地図：5系統がそれぞれ強みを分担する

DeepSeek V4

大規模MoE・低単価
self-hostコスト最良
推論・コード汎用

Qwen 3.6

サイズ展開が豊富
汎用・多言語に強い
エッジ〜サーバ

Kimi K2.6

超長文脈・エージェント
sub-agent並列向き
MoEで局所展開

GLM 5.1

MITライセンス
商用・FTの自由度大
Arena Elo首位

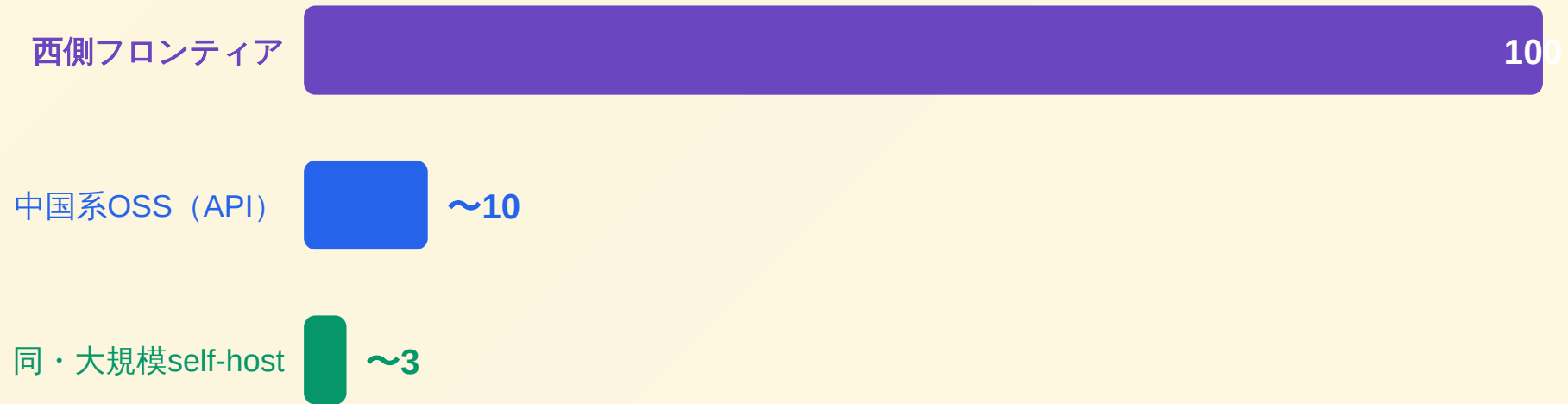
Llama 系

エコシステム最大
ツール・派生が豊富
西側OSSの基盤

コストは10～100x安：中国系OSSが西側フロンティアの1/10～1/30

“同等品質なら per-token 単価で1桁以上の差 — 高トラフィックほど自前化の回収が早まる”

出力100万トークンあたり相対コスト（フロンティア=100）



差は 1/10 ～ 1/30。運用規模が大きいほど絶対額の削減が効く

※GPU償却・電力・運用人件費を含めた実効コストで判断すること

- 低トラフィックなら API 従量課金が有利 — 損益分岐はトラフィック量で決まる
- 単価差だけでなく、GPU 償却・運用体制・SLA を含めた実効コストで比較する

MIT / Apache 2.0 が商用利用とファインチューンを後押しする

“ライセンスは「使えるか」ではなく「どこまで自由に改変・再配布できるか」で効いてくる

39

MIT / Apache 2.0

商用利用：自由
改変・再配布：自由
FTモデルの権利：明快

例：GLM 5.1

企業導入の摩擦が最小

独自OSSライセンス

商用利用：概ね可
規模・用途に条件
利用規約の確認要

例：一部Llama系

大規模利用は要精査

クローズドAPI

改変：不可
self-host：不可
ベンダーロック

従量課金

運用は最小

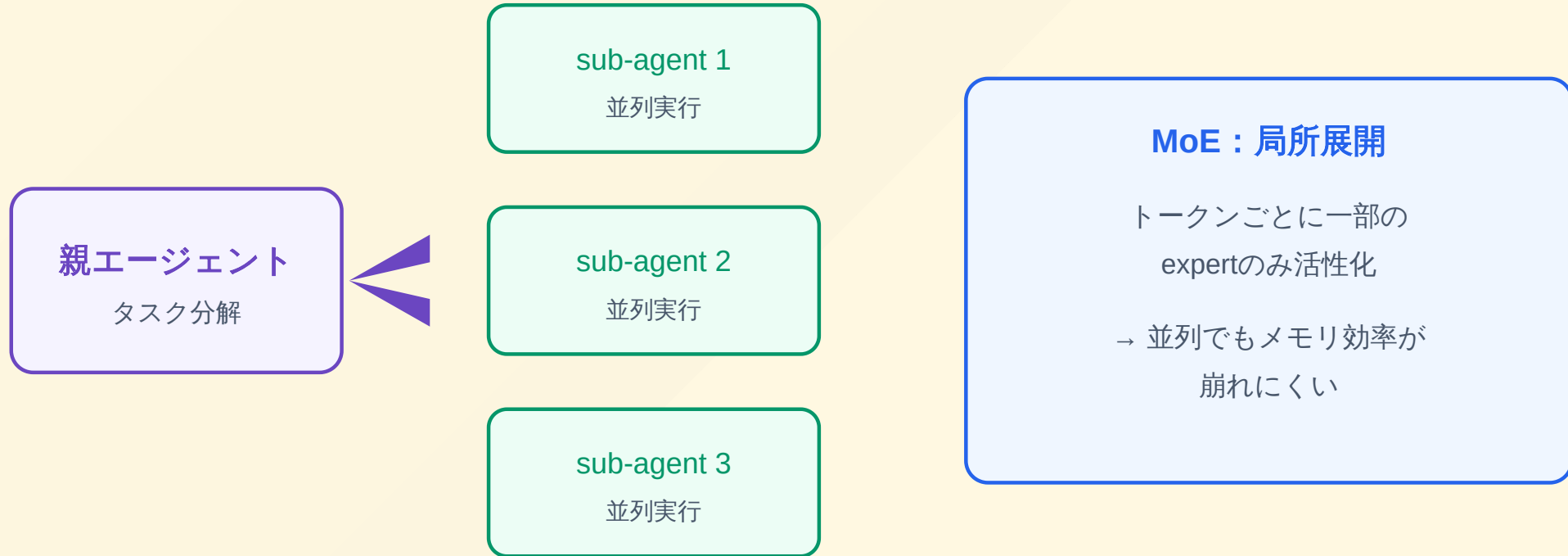
- MIT/Apache 2.0 なら社内データでのファインチューンと再配布が明快
- ライセンス精査は法務コストを含む — 導入前に必ず用途と規模を確認する

モデル別詳解

- 主要4モデルの設計思想と、self-host 時の運用上の勘所を掘り下げる

Kimi K2.6 : sub-agent並列に向き、MoEで局所展開しやすい

“超長文脈とエージェント志向 — 複数タスクを並列に走らせるワークロードで真価を発揮する”



- 長文脈を活かした計画立案と、子タスクの並列実行が組み合わせやすい
- MoE により活性パラメータが抑えられ、複数プロセスでもメモリが逼迫しにくい

Qwen 3.6 : 汎用に強いが、72B級はローカルにマルチGPUを要する

“サイズ展開が広く用途に合わせやすい一方、大型を自前で回すには相応のGPU投資が要る

”

サイズ別の展開先の目安

小型 (～7B)

単一GPU / エッジ

量子化でCPUも可

試作・オンデバイス

導入障壁：低

中型 (～32B)

単一～2GPU

量子化で1枚に収まる

本番の主力帯

導入障壁：中

大型 (72B級)

マルチGPU必須

VRAM合算が前提

高品質・高負荷用途

導入障壁：高

- 多言語・汎用タスクの安定感が強み — 用途に合わせサイズを選べる
- 72B級を常時稼働させるなら、GPU台数と電力の固定費を損益分岐に織り込む

GLM 5.1 : MITライセンスが企業ファインチューンと商用の差別化になる

“ 性能首位クラスに加えMITの自由度 ― 社内データでの改変と再配布を法務摩擦なく行える ”

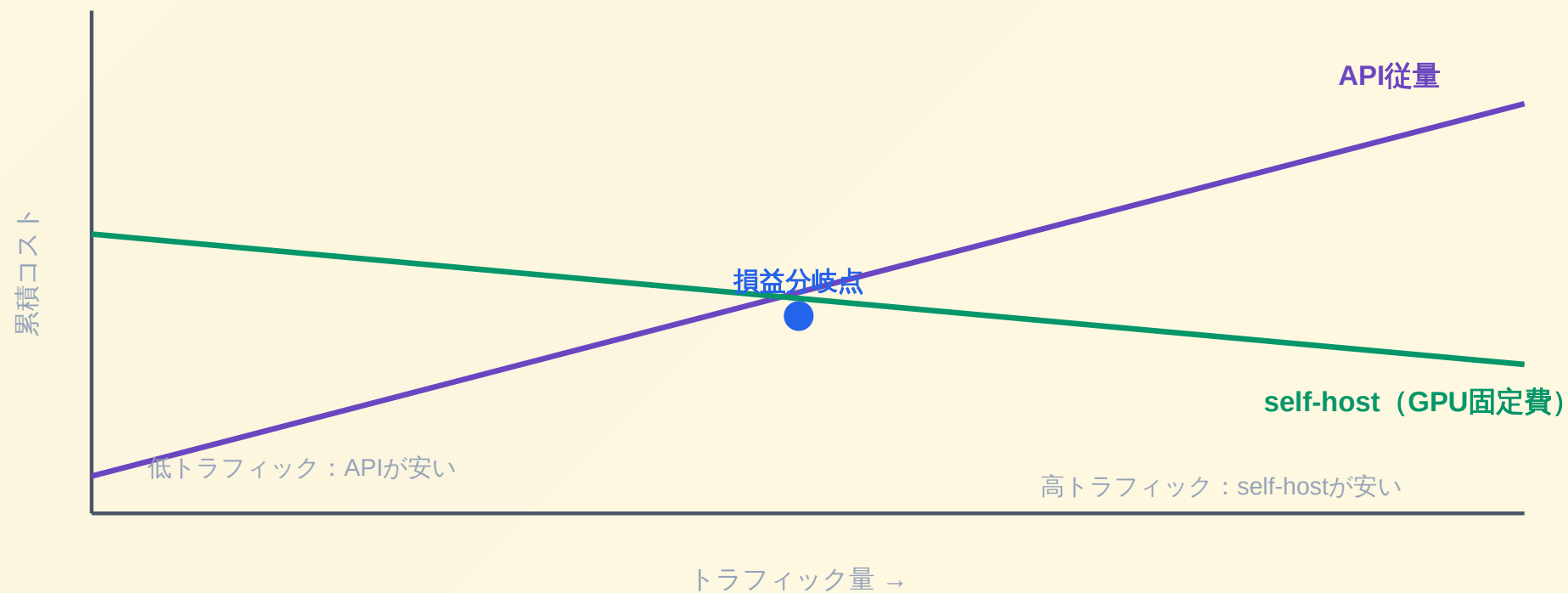


- 重み公開 + MIT により、機密データを外に出さずドメイン特化が可能
- ライセンスの自由度そのものが、他社と差がつく資産になる

DeepSeek V4 : self-host時のコスパが最も良い選択肢

“大規模MoEで活性パラメータを抑え、高トラフィックほど自前運用の回収が早い”

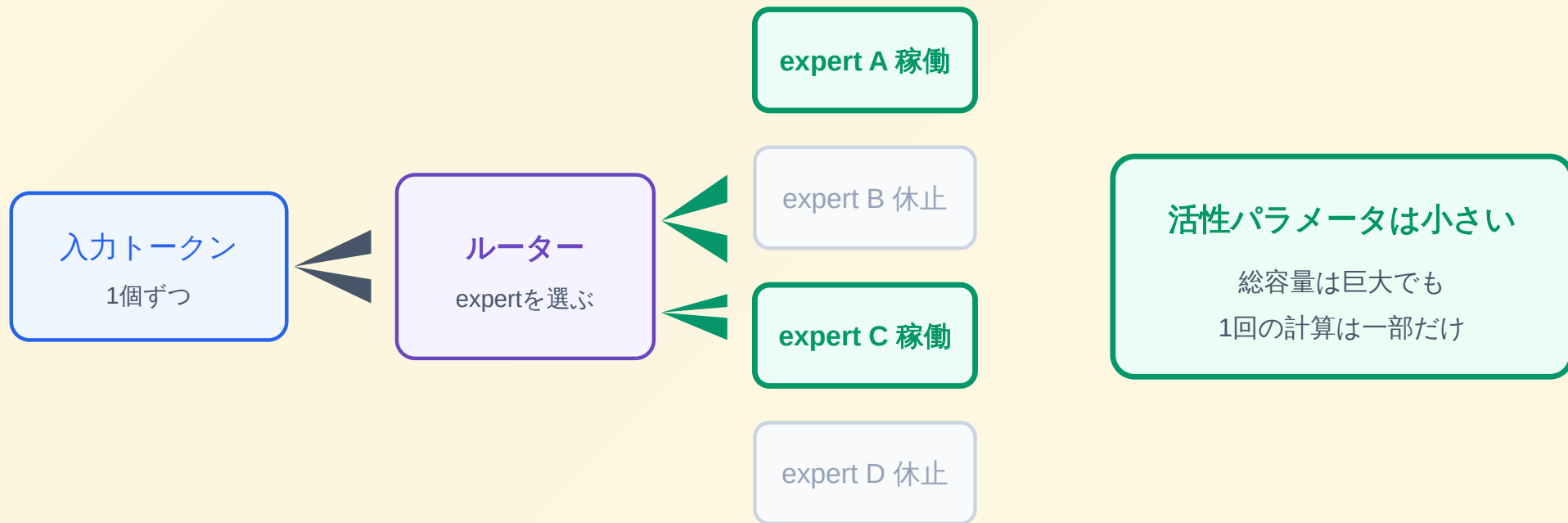
月間トークン量と累積コスト（概念図）



- 初期はGPU固定費が重い、量が増えるほど1トークン単価が下がる
- 損益分岐点を超えるトラフィックがあるかが、自前化を選ぶ唯一の条件

MoEアーキが「局所展開」を実用化する仕組み

“総パラメータは巨大でも、1トークンで動くのは一部のexpertだけ — だから省メモリで回る

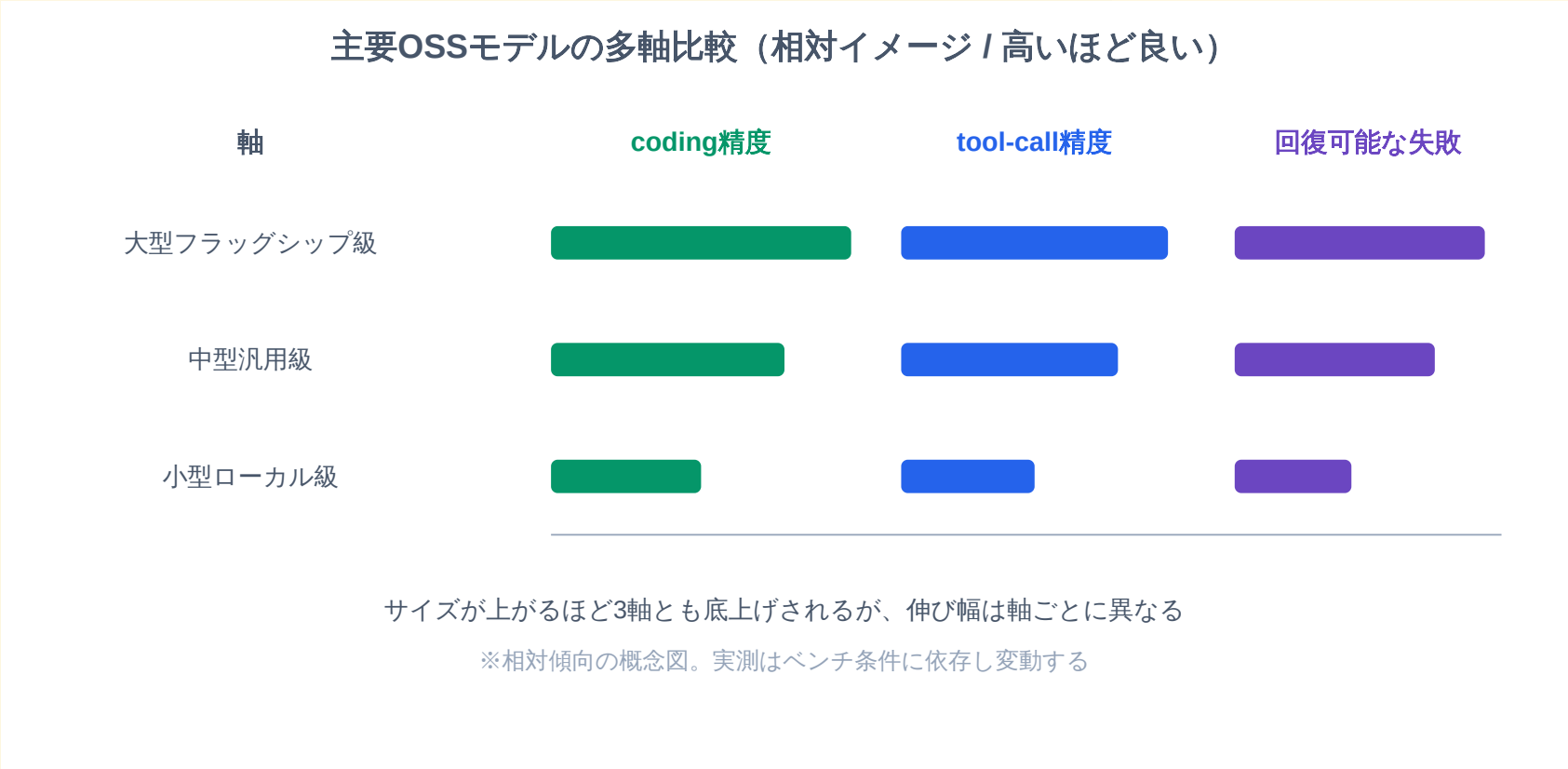


- 総パラメータ = 品質、活性パラメータ = 1トークンあたりの計算量とメモリ帯域
- この分離により、巨大モデルでも現実的なGPU構成で self-host できる

coding精度・tool-call・回復力の3軸で優劣が分かれる

“単一スコアではなく用途軸ごとの相対比較が選定の起点になる”

- 同じOSSモデルでも、coding、tool-call、失敗回復の得意分野は様々ではない



- tool-callの安定性はエージェント用途で特に効く。回復力は長時間タスクの完走率を左右する

ライセンスの寛容さが商用self-hostの自由度を決める

“Apache/MIT系は改変・再配布・商用が自由、制限付きは条件確認が必須”

- 同じ「オープン」でも、商用可否・配布条件・利用規模の制約は大きく異なる

ライセンス種類の比較

寛容型 (Apache-2.0 / MIT)

- ・商用利用：自由
- ・改変・派生モデル：可
- ・再配布：帰属表示のみ
- ・利用規模の上限：なし

→ 企業self-hostに最も適する

Qwen系やApache採用の多くがこの型

制限付き (独自ライセンス)

- ・商用利用：条件付き
- ・大規模利用に許諾要の場合
- ・出力・派生の扱いに条件
- ・許容用途の限定あり

→ 導入前に規約を精読すべき

「オープン」でも一律ではない点に注意

選定は性能×コスト×ライセンス×運用性の4軸で決める

“どれか一つの最強ではなく、4軸のバランスが自社要件に合うかで選ぶ”

OSSモデル選定の4象限



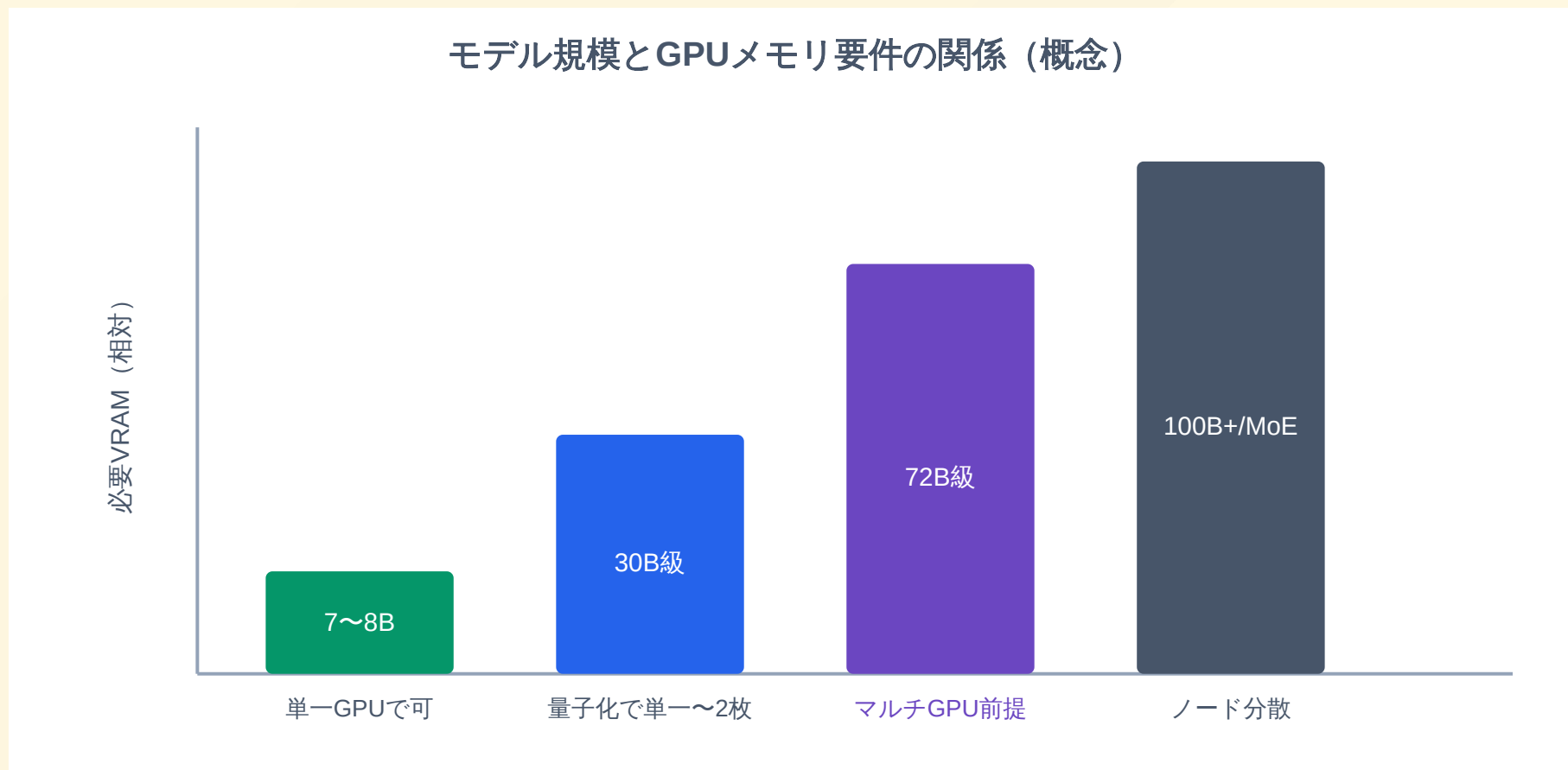
- 1軸で選ぶと後段で破綻する。4軸を要件の重み付きで評価するのが実務的

self-hostの課題

- 自前運用が抱える現実的なコストと制約を直視する

72B級モデルは単一GPUに載らずマルチGPU前提になる

“パラメータ規模が上がるほどVRAM要件が跳ね上がり分散構成が必須になる”



- 量子化（4bit/8bit）で要件は緩和できるが、精度とのトレードオフを伴う

運用コストは電力・保守・可用性を含めた総所有で測る

“ GPU購入費だけでなく、稼働し続けるための継続費が本当のコストを決める ”



量子化・バッチング・KVキャッシュが推論効率を底上げする

“ 同じGPUでもこの3手法でスループットと収容量が大きく変わる ”

主要な推論最適化の3手法

量子化

重みを低ビット化
4bit / 8bit

効果

VRAM削減・収容増

代償

精度が微減しうる

バッチング

複数リクエストを
まとめて処理

効果

GPU利用率を最大化

代償

個別遅延がやや増

KVキャッシュ

過去トークンの
中間状態を再利用

効果

長文生成を高速化

代償

追加VRAMを消費

self-hostの負担は規模・運用・技術の3点に集約される

“ GPU確保、24/365運用、推論チューニングの3つが同時に効いてくる ”

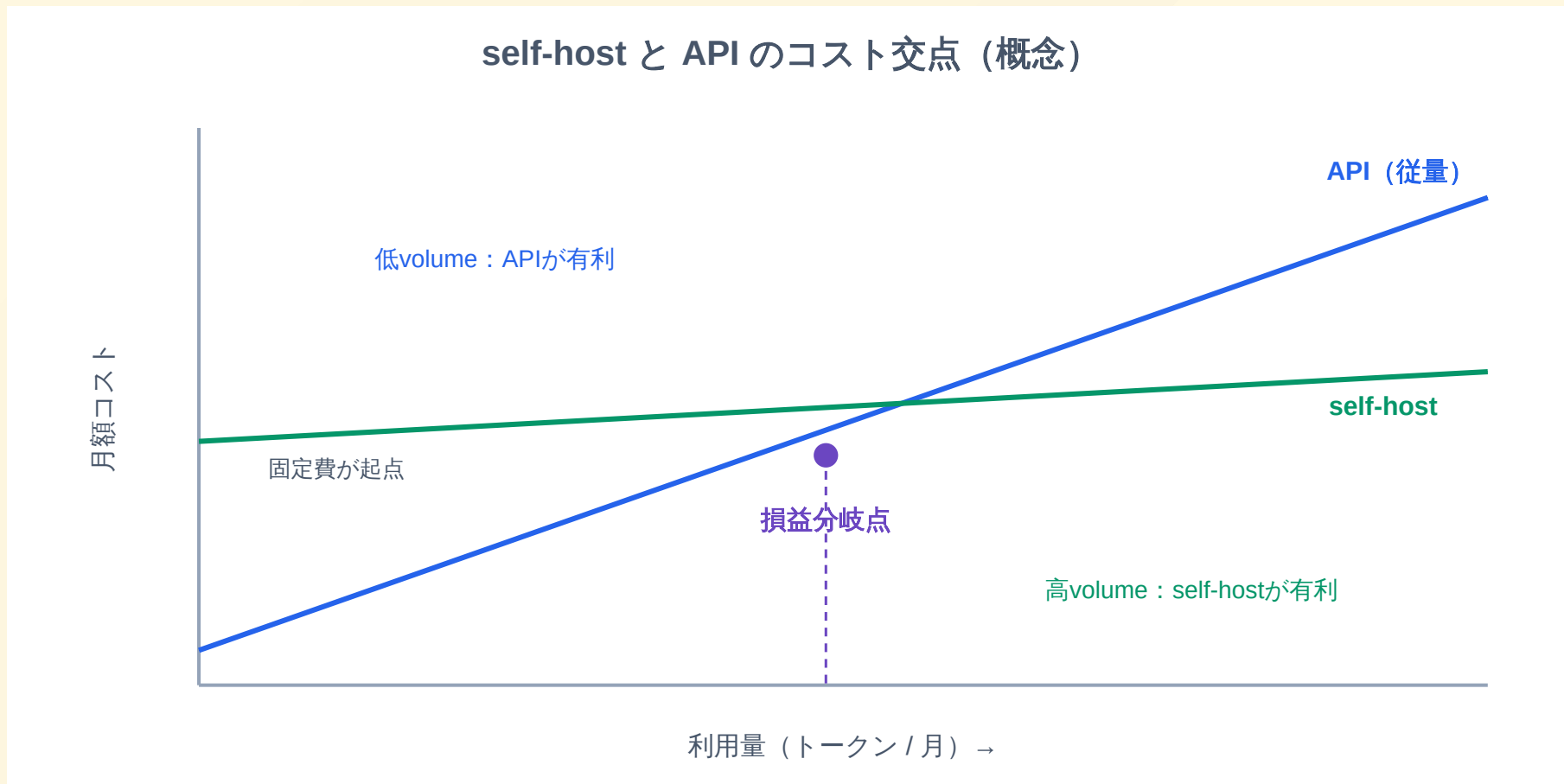
- ここまでの課題を、意思決定に効く3点へ整理する
- GPU規模：72B級以上はマルチGPU前提。VRAMと調達コストが階段状に増える
- 運用負担：電力・保守・可用性担保が継続費として積み上がり、TCOを押し上げる
- 技術要求：量子化・バッチング・KVキャッシュを使いこなす専門性が完走率を左右する
- これらを内製で吸収できるかが、self-host成否の分かれ目になる

損益判断と指針

- self-hostとAPIのどちらが得か、判断軸と実行指針を示す

高volume帯ではself-hostがAPIのコストを逆転する

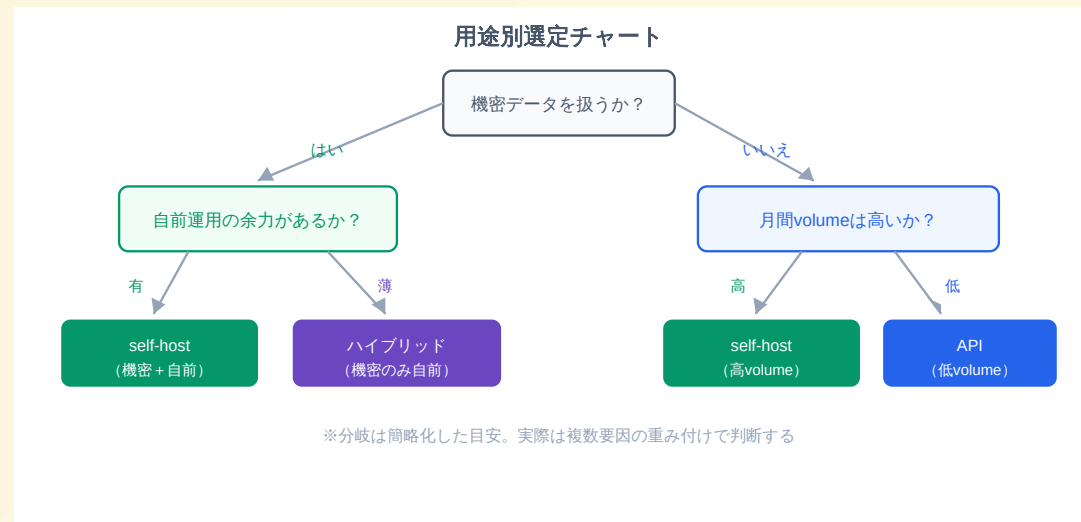
“ 利用量が損益分岐点を超えると、固定費型のself-hostが従量課金を下回る ”



- 分岐点の位置は、モデル規模・GPU調達方式・稼働率で前後する。自社の実測volumeで検証する

機密性・volume・カスタム要件で推奨構成が決まる

“ 3つの分岐を順に進めると、*self-host* / *API* / ハイブリッドのどれが適合するか見える ”

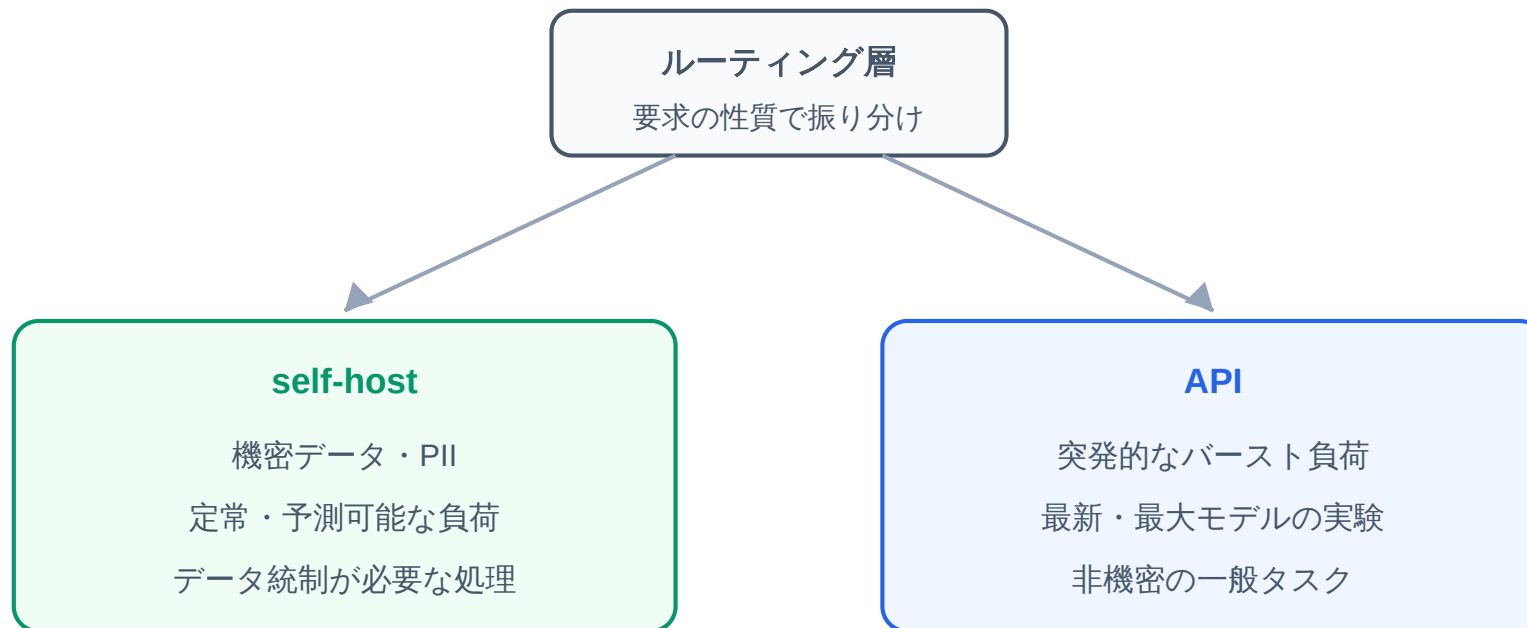


機密はself-host・バーストはAPIで賄うのが現実解

“ワークロードを性質で振り分ければ、統制と弾力性を両取りできる”

”

ハイブリッド構成の振り分け



- 統制が要る処理は自前で守り、弾力性が要る処理はAPIに逃がす。二者択一にしない

MoE選択・量子化・ルーティング・キャッシュでコストを削る

“モデル選択と運用工夫を重ねると、同じ処理量でも総コストが下がる”

- コスト最適化は単発の施策ではなく、複数レバーの積み上げで効く



小さく検証し、成功パターンを段階的に本番へ広げる

“PoC→限定本番→拡大の3段で、リスクを抑えつつ内製力を蓄積する”

- 導入は一足飛びにせず、学習しながら投資を増やすのが失敗しにくい
- ステップ1：非機密の限定用途でPoC。API併用で素早く価値を検証する
- ステップ2：損益と要件が見合う処理からself-host化。運用監視・回復手順を固める
- ステップ3：ハイブリッドで拡大。機密は自前、バーストはAPIへ振り分けを標準化する
- 3アクション：①損益分岐volumeを実測 ②ライセンス規約を精読 ③運用体制と撤退条件を先に決める

まとめ：性能から損益分岐へ

“OSS LLMは「使えるか」から「どう賢く使い分けるか」の段階に入った”

- OSS LLMは「使えるか」から「どう賢く使い分けるか」の段階に入った
- 性能・コスト・ライセンス・運用性の4軸で、自社volumeに合った構成を選ぶ
- 機密はself-host、バーストはAPI—二者択一ではなくハイブリッドが現実解

参考文献・出典

- 以下の公開情報を参照
- Kimi K2.6 and Qwen 3.6: Closing the Frontier Gap – MindStudio: <https://www.mindstudio.ai/blog/kimmy-k2-6-qwen-3-6-open-source-frontier-models>
- Best Open-Source LLMs for Agentic Coding in 2026 – MindStudio: <https://www.mindstudio.ai/blog/best-open-source-llms-agentic-coding-2026>
- Open-Weight LLM Showdown 2026 – Wavect: <https://wavect.io/blog/open-weight-llm-comparison-2026/>
- Best Open Source Self-Hosted LLMs for Coding in 2026 – Pinggy: https://pinggy.io/blog/best_open_source_self_hosted_llms_for_coding/